PTUT — FINAL YEAR PROJECT

Weekly Progress Report · Week 10 of 12

# TaxWise Freelance

Automated Tax Deductions & Invoicing Platform

## Automated Testing & Security Audits


STUDENT
**Muhammad Sohaib Ramzan**

ROLL NUMBER
**24-ST-004**

SUPERVISOR
**Dr. Zaigham Mushtaq**

DEPARTMENT
**Software Engineering Technology**

SECTION
**A**

PROJECT ID
**PTUT-PRJ-042**

DOMAIN
**FinTech, Personal Wealth & Micro-Finance**


Source code: github.com/SohaibRamzan01/taxwise-freelance

# 1. Summary

This week focused on **quality and security**. The automated test suite now covers **99.6% of the core logic** (the target was 70%), every API route was reviewed against the **OWASP Top 10**, a **static security scan** was run, and dependency vulnerabilities were checked: **0 known vulnerabilities in the packages that ship with the app**.

**296**
automated tests passing

**99.6%**
line coverage on core logic (target 70%)

**0**
known vulnerabilities in production packages

| Task from the plan | Status |
|---|---|
| Automated test suite with ≥ 70% coverage on core logic | Done — 99.6% lines, 96.3% branches |
| Static security scans | Done — ESLint security rules: 0 errors, 27 warnings reviewed by hand |
| Patch dependency vulnerabilities | Done — 0 in production packages; 5 in a development-only tool documented |
| Sanitise all inputs against OWASP Top 10 | Done — route-by-route review |

# 2. Test coverage

Coverage is measured on `src/lib` (the core logic) and **enforced**: the test run fails if any value drops below 70%.

| Measure | Result | Target |
|---|---|---|
| Lines | **99.62%** | 70% |
| Statements | **99.20%** | 70% |
| Functions | **98.77%** | 70% |
| Branches | **96.32%** | 70% |

| Test group | Examples |
|---|---|
| Tax engine | Hand-checked $100,000 example, every bracket boundary for all filing statuses, safe harbour, rounding |

| Test group | Examples |
|---|---|
| Data separation | User B cannot read, change or delete user A's expenses, income, clients, invoices or payments |
| Access control | Normal users get 403 on admin routes; disabled users are locked out |
| Inputs | Bad JSON, too-large requests, unknown fields, invalid dates and amounts are rejected |
| OCR & files | Real OCR on sample receipts; fake images (wrong file signature) are rejected |
| AI fallback | Gemini errors, timeouts and bad answers fall back to rules |
| Background job | Safe to run twice; rejected without its secret |

## 3. OWASP Top 10 (2021) review

| Risk | How TaxWise handles it |
|---|---|
| A01 Broken Access Control | Every query filtered by the logged-in user's ID; admin role checked in the database; origin check on changes |
| A02 Cryptographic Failures | bcrypt (cost 12) passwords; signed JWT (HS256) in httpOnly, Secure cookies; HTTPS + HSTS |
| A03 Injection | zod checks every input; Prisma uses safe parameters; no raw HTML rendering; CSV formulas escaped |
| A04 Insecure Design | Server calculates all totals; paid invoice → exactly one income row (transaction + unique key) |
| A05 Security Misconfiguration | Content-Security-Policy, X-Frame-Options DENY, nosniff, referrer policy; generic error messages |
| A06 Vulnerable Components | Locked dependency versions, `npm audit` in the review, CI on every push |
| A07 Authentication Failures | Login rate limit (5 tries / 15 min), same error for wrong email or password, disabled accounts blocked |
| A08 Data Integrity Failures | CI installs from the lockfile; AI answers schema-checked; OCR language data bundled (no downloads) |
| A09 Logging & Monitoring | Audit log for logins, deletes, role changes and job runs; no passwords or tokens in logs |
| A10 Server-Side Request Forgery | The server never fetches user-supplied URLs; only one fixed Google API address |

## 4. Static scan and dependency audit

| Check | Result |
| --- | --- |
| ESLint security plugin | **0 errors**, 27 warnings — each reviewed by hand (object lookups with fixed keys, bounded number patterns, fixed file paths in scripts) |
| `npm audit` (production packages) | **0 vulnerabilities** |
| `npm audit` (all packages) | 5 "high" in `braces`, used only by the development lint tool; no non-breaking fix exists yet; it is not shipped to users |
| Secret scan | No passwords, keys or tokens in the code or Git history; `.env` files are ignored |

## 5. Input protection added this week

- **Request size limits** while reading JSON (stops huge requests early).
- **Upload checks** by real file signature (magic bytes), size limit, and OCR time limit.
- **Number limits** in the receipt parser and money parser so unusual text cannot slow the server down.

## 6. Challenges and solutions

| Challenge | Solution |
| --- | --- |
| A development-only package has an advisory with no compatible fix | Documented and tracked; it is not part of the deployed app |
| In-memory login rate limit is per server instance | Documented as a known limit; a shared store is recommended for large-scale use |
| Tests that need a real database are slow | They run in CI with a PostgreSQL container; locally they are opt-in |

## 7. Plan for next week (Week 11)

- Deploy the app to a live public URL with SSL (Vercel + Neon).
- Run the GitHub Actions CI pipeline on every push.
- Verify the production health checks with a live end-to-end test.